

Acropolis: A Portable, AI-Driven Semantic Search System for HPC

Rajni Parshuram Pawar, Luke Logan, Jaime Cernuda Garcia, Xian-He Sun, Antonios Kougkas
 Illinois Institute of Technology, Chicago, IL, USA
 rpawar4@hawk.illinoistech.edu, {llogan, jcernudagarcia, sun, akougkas}@illinoistech.edu

Abstract—LLM agents are now a primary interface to data repositories, but they locate files by issuing speculative regex queries through `glob`, `grep`, and file reads—a guess-and-check loop expensive in tool calls and tokens that ignores the metadata embedded in scientific file formats. We present *Acropolis*, a semantic search system that augments a tiered filesystem with adaptive-depth content indexing and exposes it to agents through a typed natural-language interface. *Acropolis* makes four contributions: a transparent file-summary operator that runs inside the I/O ingest path and is designed to amortize its dominant LLM cost through content-addressable idempotency; a pluggable search-backend abstraction surfacing lexical, vector, and graph engines as a deployment-time choice, evaluated empirically; an MCP server that exposes the resulting search surface to any compatible agent; and a scientific-data evaluation that quantifies the resulting agent-side savings on a 70-query CMIP6 retrieval benchmark, where an agent given *Acropolis* alongside its shell tools matches the shell-tool baseline on retrieval accuracy while substantially reducing both tool/backend calls per query and LLM token consumption.

I. INTRODUCTION

LLM agents are now a dominant consumer of filesystem data. Modern agent IDEs (Claude Code, Cursor, Sourcegraph Cody) and domain-specific scientific agents such as ChemCrow [1] and The AI Scientist [2] routinely retrieve files on behalf of human users, but the POSIX interface they inherit gives them almost nothing semantically useful to work with. The dominant agent retrieval pattern is borrowed wholesale from how a human navigates a filesystem at the shell: `glob` a regex against file names, `grep` a regex against content, `Read` a candidate, repeat. Applied to an LLM in the loop, this guess-and-check pattern is slow and expensive. It can take several tool calls and tokens to analyze the contents of files with this approach.

There are two general approaches to closing this gap. The current standard is to copy data into a separate vector database or knowledge graph such as Chroma [3] and run semantic search there. For scientific data this copying is infeasible, since the corpora are too large to duplicate into a secondary index and the storage layer ends up reading every byte twice, once to write it and again to feed the external indexer. The alternative is to bring semantic search directly into the storage layer where the data already lives, so that indexing rides on the I/O the system already performs. A wave of recent products takes this route, including VAST Data [4], WEKA NeuralMesh [5], and IBM Storage Scale Content-Aware Storage [6], alongside bolt-on variants such as DDN HyperPOD [7] that run a retrieval service next to the storage rather than inside it; all of these are closed source and tightly coupled to specific hardware and

software stacks, most of them sold as pre-integrated NVIDIA-GPU appliances. The deeper problem is that each one binds the storage layer to the vendor's own AI stack. The summarizer that interprets file content and the embedding that turns it into vectors are confined to the vendor's bundled model catalog, the vector index is the vendor's proprietary engine reached through a vendor API, and an operator cannot point the search at the keyword or graph infrastructure their site already runs. Because the index is built around one embedding, the data it produces is only meaningful to that vendor's own search path, which confines users to the system that produced it and means an operator who outgrows the bundled stack has to re-embed the entire corpus to move.

Building a portable semantic search directly in the storage layer raises three challenges, each addressing one axis of vendor lock-in. The first is *transparent context assimilation*: the storage layer must extract machine-readable meaning from arbitrary file formats—HDF5, netCDF, source code, plain text—without the application calling into it explicitly. The second is *selecting a search engine*: it is not immediately clear which kind of retrieval system is best for querying automatically generated summary data. Several families exist—lexical, vector, and graph—each with its own tradeoffs in indexing cost, query semantics, and scaling behavior, and the right choice for a workload of this shape is an empirical question. The third is *an open agent-facing interface*: a typed natural-language entry point in the protocol modern agents already speak, so the same data is reachable from any MCP-compatible client rather than confined to one vendor's API.

We present *Acropolis*, which implements a semantic search filesystem using the Context Layer for I/O (CLIO), an existing multi-tiered I/O system. The contributions of *Acropolis* are:

- **C1** (transparent assimilation): a transparent ingest-time summary operator that runs inside the CLIO I/O path, with per-format extractors (HDF5 / netCDF on the scientific side, head/tail content sampling for source-tree corpora), adaptive depth (L0 path, L1 path+metadata, L2 path+metadata+content+LLM summary), and content-addressable idempotency that short-circuits the LLM call on re-ingest of unchanged files.
- **C2** (search engine selection): a `QueryEngine` interface with four engines selectable per deployment—BM25 (in-process), Elasticsearch, Qdrant, and Neo4j, with Elasticsearch and Neo4j each exposing keyword, vector, and RRF hybrid retrieval modes—and a systematic study of their impact on retrieval performance over automatically

TABLE I: Related systems vs. Acropolis along the five design dimensions: embedded indexing, format-aware extraction, pluggable retrieval engine, open agent interface, and adaptive indexing depth.

System	Embedded	Format	Plug	Open	Depth
Chroma context-1	—	—	—	—	—
VAST Data	✓	—	—	—	—
WEKA NeuralMesh	✓	—	—	—	—
IBM Storage Scale CAS	✓	✓	—	—	—
DDN HyperPOD	—	—	—	—	—
LSFS	✓	—	—	—	—
Acropolis	✓	✓	✓	✓	✓

generated summary data.

- **C3** (agent-shaped harness): an MCP [8] server that exposes a single `semantic_query` tool to any MCP-compatible agent, replacing the speculative `glob/grep/Read` loop with one typed call that returns ranked candidates with summaries on the first response.
- **C4** (an evaluation on a 70-query CMIP6 scientific benchmark) that quantifies Acropolis’s effect on agent tool-call and token consumption for file retrieval.

II. BACKGROUND AND RELATED WORK

The discoverability gap shows up in two settings the same way. In scientific computing, data lives in parallel filesystems optimized for bandwidth and capacity, not discoverability: directory layouts and file names are chosen by the producing pipeline and rarely revisited, and downstream consumers locate files with `find`, `glob`, and `grep`. Scientific data catalogs such as iRODS [9] and Globus [10] carry structured metadata alongside data, but the metadata they index is whatever the producing pipeline chose to annotate, so a file the pipeline did not describe is invisible to the catalog. Format-aware extractors built into HDF5 [11] and netCDF expose structural attributes (dataset names, shapes, dtypes, units) but describe how a file is shaped rather than what it means, and they are not searchable by natural-language query. In source-code repositories, agent IDEs (Claude Code, Cursor, Sourcegraph Cody) face the same shape of problem against a different format: the structural metadata is parse-tree shaped (symbols, call graphs, includes) rather than dataset-shaped, but the gap between what an LLM agent wants to ask (“which file implements the BM25 backend?”) and what the storage layer can natively answer (path globs and byte ranges) is identical. The discoverability gap therefore has two distinct causes that are independent of file format: an interface mismatch (POSIX has no notion of meaning) and an architectural mismatch (the layer that sees every write is not the layer that handles queries). The architectural mismatch is the more fundamental of the two: any external indexing service has to reconstruct, after the fact, knowledge the storage layer already had at write time. Recent systems addressing this gap fall into three categories.

External Semantic Search. Chroma context-1 [12] packages a gpt-oss-20b fine-tune with a retrieval harness into a complete agent-facing search system. The system lives outside the storage layer: a producing application writes files to storage as before, and a separate Chroma index reads those files back to

build its vector representation. Because the corpus has to be re-read out of storage before it can be indexed, large scientific corpora pay their I/O cost twice—once to write and once for indexing. Section IV uses Chroma context-1 as a direct external comparison point.

Closed-Source Embedded Semantic Search. A wave of 2025 storage products embeds semantic search in the storage layer itself. VAST Data [4] introduces event-driven vector search with NVIDIA NIM embedding agents triggering on write, a DASE shared-everything architecture, and row/column ACL enforcement on the resulting vector index. WEKA NeuralMesh [5] bundles its parallel filesystem with an auto-vectorization pipeline (parsing → chunking → embedding → vector DB) that fires on every write and claims UID/GID ACL enforcement on queries. IBM Storage Scale Content-Aware Storage [6] is the closest commercial system to the Acropolis thesis: a custom vector database backed by IBM Granite embeddings, Docling parsing for PDFs and Office documents, and automatic propagation of Storage Scale filesystem ACLs to vector search results. DDN HyperPOD [7] takes the opposite architectural choice within the same product space, running NVIDIA NeMo Retriever as a bolt-on service alongside the storage rather than inside it. All four are closed source and tightly coupled to specific hardware and software stacks; each binds the storage layer to a vendor-specific summarizer, embedding model, vector index, and query API, so an operator who outgrows the bundled stack must re-embed the entire corpus to migrate.

Open-Source Embedded Semantic Search. LSFS [13] (ICLR 2025) is the closest open-source position to Acropolis. It embeds file contents into vectors via LlamaIndex and provides `semantic_retrieve` and `keywords_retrieve` syscalls that any application can invoke from within the operating system. LSFS demonstrates the syscall-level semantic-search concept but operates on text-only corpora at toy scale (10–40 files), implements only basic lock/unlock permissions, and provides no indexing-depth control, no scientific-format awareness, and no agent-facing tool surface. It also commits to a single retrieval engine (the LlamaIndex vector index) rather than allowing the operator to plug in an existing keyword or graph backend.

In summary, and as Table I makes explicit, every prior system covers only part of the design space: external semantic-search systems pay the I/O cost twice; closed-source embedded systems bind the storage layer to a single vendor stack; and the closest open-source embedded system fixes a single retrieval engine and addresses only text files at toy scale. Acropolis is the first to combine embedded indexing, format-aware extraction across scientific and text-shaped data, a pluggable retrieval engine spanning lexical, vector, and graph, an open MCP query interface, and operator-controlled adaptive indexing depth inside a single tiered-storage system.

III. DESIGN

Acropolis brings semantic search to CLIO [14], an existing multi-tiered I/O system whose storage runtime hosts the relevant logic. Of the paper’s four contributions, this section describes the three design contributions: (C1) transparent

context assimilation that runs inside the I/O ingest path and attaches a searchable description to every ingested file; (C2) a pluggable search interface that lets a deployment pick the retrieval engine without changing client code; and (C3) an MCP server that exposes the search surface as a single tool any modern agent can invoke. The fourth contribution (C4), the 70-query CMIP6 evaluation, is reported in Section IV. We describe the integrated platform first, then each design contribution in turn.

A. High-Level Architecture

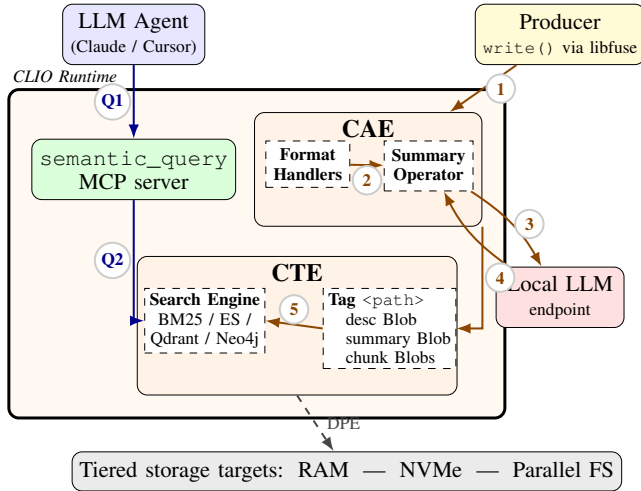


Fig. 1: High-level architecture demonstrating the flow of automatic summarization during write operations.

The Acropolis platform is layered on top of CLIO’s multi-tiered storage runtime. Two CLIO modules host the relevant logic—the Context Assimilation Engine (CAE), which runs the format-aware extractors and the summary operator, and the Context Transfer Engine (CTE), which provides tiered blob storage and houses the pluggable search engine that holds the searchable index—together with an MCP server that exposes the search surface to the calling LLM. All three components run inside the same CLIO runtime, so cross-module calls are in-process IPC rather than network RPCs.

The unit of storage in CTE is a *Blob*: a named binary payload that the storage runtime tiers, replicates, and durably persists. A *Tag* is a named grouping over a set of related Blobs; it captures the relationship between Blobs that belong together (e.g., the original file content, its extracted description, and its LLM summary) without itself carrying any payload. Acropolis allocates one Tag per ingested file and writes everything it learns about that file (the file’s extracted description, its LLM summary, and any content chunks) as sibling Blobs inside that Tag. The search engine is keyed on the Tag identifier, so a single search hit dereferences to the full collection of Blobs the system has on that file.

A typical end-to-end lifecycle proceeds as follows. The producing application mounts CTE as a POSIX-compatible filesystem through a libfuse adapter and writes files into it as it

would to any other filesystem; no application code changes are required to participate in Acropolis. On each `write()`, the libfuse adapter routes the modified bytes into CTE under a Tag named after the file’s path, and consults CAE’s deployment-time configuration—a list of path patterns mapped to per-format extractors and summarization operators—to decide whether the write triggers indexing. If the path matches, CAE schedules a background ingest task for the chunk that was modified: the appropriate format-aware extractor (§B) reads the chunk and writes a `description` Blob into the Tag; the summary operator (§C) then reads that description, calls the LLM, and writes a `summary` Blob into the same Tag. CTE places each Blob on the storage tier chosen by the Data Placement Engine, which considers per-device capacity, throughput, and load to decide whether the Blob lands in RAM, on NVMe, or on the parallel filesystem; large Blobs are physically split across multiple devices when necessary. Crucially, the `write()` call returns as soon as the bytes are durable in CTE; the indexing pipeline runs asynchronously so that a workflow producing a file in many small chunks can have its modifications aggregated into one re-indexing pass rather than paying the summarizer cost on every chunk. Once the description and summary Blobs are durable, CAE notifies the configured search engine (§D), which inserts an index entry keyed by the Tag identifier with text drawn from the description and (at L2) the summary. The Tag is now both stored in CTE and searchable through the configured engine. At query time, the calling agent invokes the `semantic_query` MCP tool (§E), which queries the search engine and returns the ranked top- k Tags together with their summaries.

B. Format-Aware Context Extraction

The first contribution moves data understanding out of the application and into the ingest path itself, so every byte entering CTE is paired with a machine-readable description of what it is at the moment it is written. Locating this work at ingest exploits the fact that the storage layer is the one place that necessarily touches every byte exactly once, so the marginal cost of attaching semantics there is bounded by the cost of the extraction itself; an offline indexer would have to re-read every file from a tier whose I/O cost was already paid at write time.

When the libfuse adapter receives a write, it dispatches the modified bytes to a format-aware handler selected by the file’s extension. HDF5 writes go through a handler that walks the file structure with `H5Lvisit` and processes each dataset as its own sub-task; netCDF and other scientific formats follow the same pattern with their own handlers; opaque binaries fall through to a generic handler that reads the modified byte range. The handler emits a `description` Blob into the file’s Tag, containing the file path and whatever text the handler extracted. A new file format is added by registering a new handler keyed on the extension; subsequent writes are dispatched to it automatically. Repeated modifications to the same file are aggregated into a single re-indexing pass rather than re-running per chunk.

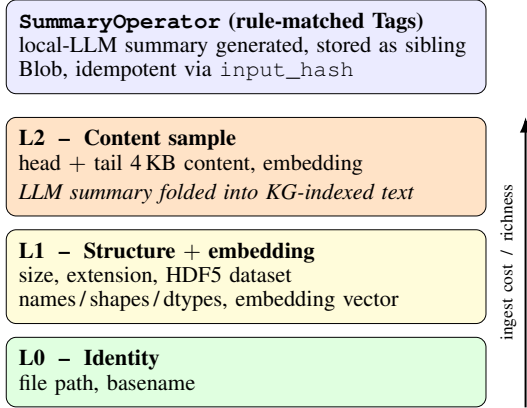


Fig. 2: Adaptive index depth. The level controls what text is indexed against a Tag: L0 only identity, L1 adds structural metadata and an embedding, L2 additionally folds in a head/tail content sample and the LLM summary. The `SummaryOperator` runs on the (Tag, Blob) pairs that match a configured rule and stores its output as a sibling Blob; the summary is included in the searchable index only at L2, but remains retrievable on demand at any level.

C. Summary Operator

The summary operator turns each Blob produced by the extractor into a query-friendly natural-language summary. The operator is configured declaratively: a deployment-time specification lists rules that map patterns over Tag and Blob names to summarization parameters, and the operator runs only on (Tag, Blob) pairs that match a rule. Each rule selects which model produces the summary, with what context length, and at what target output length. Acropolis does not commit to a specific LLM or inference provider; the rule decides.

Acropolis exposes a three-level `IndexDepth` setting illustrated in Figure 2. L0 indexes only the file’s path; L1 adds an extractor-produced description (for HDF5: dataset names, shapes, dtypes, attributes; for source code or text: structural metadata) together with an embedding of the resulting text; L2 additionally embeds a head/tail content sample and is the only level at which the LLM-generated summary is folded into the indexed text. When a rule produces a summary it is stored as a sibling Blob in the Tag, so it remains retrievable on demand at any level; depth only controls whether it participates in the index.

A single configuration entry specifies what to label, how deeply to label it, and which model to label it with:

```
summarize:
- tag_re:      "*.txt"
  blob_re:     "*"
  depth:       [L0, L1, L2]
  model:       <name of LLM to use>
  endpoint:    <URL of inference provider>
  context_length: 32k
  summary_length: 8
```

The `tag_re` and `blob_re` fields select which (Tag, Blob) pairs the rule applies to; `depth` lists which index

levels the summary participates in; model and endpoint specify the inference provider; `context_length` bounds the input the operator sends; and `summary_length` specifies the target output length (in sentences). Multiple rules can coexist, with the first match winning; (Tag, Blob) pairs that match no rule are not summarized. An operator can therefore ask for short summaries on configuration files, longer summaries on documentation, and skip summarization entirely for opaque binaries or scratch data. Per-file overrides via extended attributes (`user.acropolis.depth` on a file, `user.acropolis.depth_recursive` on a directory) take precedence over the configuration, so an entire ingest campaign can be retuned without touching the producing application or restarting the runtime.

Every operator invocation is idempotent. An `input_hash` computed over the tuple (input Blob bytes, configuration entry, operator version) is stored alongside the summary Blob, and a re-run that finds a matching hash short-circuits without invoking the model. The pipeline therefore behaves as a content-addressable cache, which matters because the model call is the dominant per-file ingest cost (Figure 3); steady-state re-ingest of an unchanged corpus collapses to the cheap extraction and indexing stages.

D. Pluggable Search Backends

Once every Tag carries a description and (at L2) a summary, the question of where to index that text for retrieval is forced. Different deployment contexts call for different retrieval engines: some sites already operate an Elasticsearch cluster they want to reuse; others cannot accept any heavyweight external dependency and need a pure in-process baseline; sites that want to support graph-shaped multi-hop reasoning need a graph store. The second contribution therefore decouples the search engine from the storage substrate through a `QueryEngine` interface that the rest of Acropolis programs against.

A `QueryEngine` implementation exposes four operations: `Add(TagId, text)` inserts an index entry for a Tag with the text drawn from its description and (at L2) summary Blobs; `Update(TagId, text)` replaces the entry when a Tag’s contents change; `Remove(TagId)` drops the entry; and `Search(query, top_k)` returns ranked Tag identifiers with scores. The engine is selected per deployment by a single configuration key. Ranking, scoring, embedding-model choice, and index-time hyperparameters all stay on the engine side; the rest of Acropolis sees only these four calls. We currently provide four implementations: a pure in-process BM25 inverted index with no external dependencies; an Elasticsearch engine supporting keyword (BM25), vector (HNSW kNN), and Reciprocal Rank Fusion hybrid modes against an external cluster; a Qdrant engine that serves cosine-similarity queries over a dense embedding; and a Neo4j engine that stores each Tag as a graph node exposing the same three modes (keyword, vector, RRF). The interface accepts further engines (Chroma, Milvus, Weaviate, GraphRAG-style stores) as drop-in additions. Acropolis does not aggregate results across engines: a single engine is selected per deployment and serves every query, and § IV reports per-engine accuracy

so the operator can pick the engine that matches the expected query mix.

E. Agent-Facing MCP Server

Even with descriptions stored and a search engine available, the calling LLM cannot benefit unless the search surface is exposed in the call shape it already speaks. Today, an LLM agent that needs a file reaches for the same primitives a human would—`glob`, `grep`, and a series of file reads—driven by speculative regex guesses. Each miss costs both a tool-call round trip and the token bill for the failed response. The third contribution exposes the search surface as a single MCP [8] tool, `semantic_query(prompt, k)`, that any MCP-compatible agent can invoke. The MCP server is a thin adapter: it accepts a natural-language query, calls the configured QueryEngine, and returns a ranked top- k result with paths, scores, and (at L2) summaries. Modern agents (Claude Code, Cursor, Sourcegraph Cody) discover and call the tool naturally through their existing MCP integration with no custom glue; §IV-G shows Claude invoking `semantic_query` on every benchmark query without being directed to.

IV. EVALUATION

We evaluate Acropolis on a single CLIO mount populated to resemble a typical production filesystem: a 67 GB collection of 149 CMIP6 climate-model outputs in netCDF/HDF5 (drawn from a public cloud mirror [22]), an HPC systems source-code repository (1,886 files of build files, headers, sources, tests, configuration, and documentation), and thousands of additional files of mixed modalities—further source-code repositories, additional scientific outputs, plain-text documentation, configuration, and build artifacts. §B measures per-file ingest cost on the source-code files because their range of formats exposes the three index depths cleanly; the CMIP6 outputs are the workload Acropolis’s design targets and the substrate for all retrieval results in §§C–F, with every retrieval query disambiguating the ground-truth file against the full populated mount rather than against an isolated benchmark corpus. The CMIP6 query workload comprises 70 content-oriented questions used across all retrieval results in §§C–F. All measurements run on NCSA Delta [23], an A100-class HPC system, serving locally hostable summarizers through Ollama [25]; unless noted otherwise, the summarizer is Qwen2.5-Coder 14B [24].

A. Experimental Methodology

The agent-side experiments use Claude Code as the calling LLM, with Acropolis’s `semantic_query` MCP tool registered alongside the standard shell tools (`Glob`, `Grep`, `Read`, `Bash`); we record the tool sequence, the total call count, the path the agent returns, and a token snapshot from Claude Code’s `/cost` command. The per-level retrieval evaluation of §C fixes the call structure by experimental design, issuing one `semantic_query` per (backend, level, query) cell and scoring a hit when the ground-truth file appears in the top- k , while the agent comparison of §G places no constraint on tool choice. An offline grader scores each returned path against a per-query list of accepted substrings, matching against the provenance string encoded in each CMIP6 filename (variable,

model, experiment, member, grid, time), so scoring is mechanical and reproducible from the recorded JSON without re-running the agent.

B. Per-Level Ingest Cost

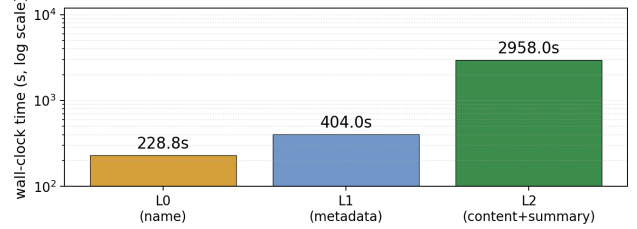


Fig. 3: Total wall-clock time (log scale) to ingest 1,886 source-code files from the populated CLIO mount (§A) at each adaptive index depth (Qwen2.5-Coder 14B summarizer at L2). L0 indexes the path only, L1 adds structural metadata and a dense embedding, and L2 additionally runs the per-file LLM summary. The L1 → L2 step inflates total ingest time by roughly 7× because the summary is a per-file forward pass through a 14B-parameter model; metadata extraction and search-engine indexing together stay under 7% of total ingest cost at every level.

The adaptive index depth of Section III, C1 is the operator’s primary cost knob, and the cost is governed almost entirely by whether the per-file LLM summary runs at all. Figure 3 reports total wall-clock ingest time at each depth across the 1,886 source-code files of the populated mount: L0 takes 228.8s, L1 takes 404s (a 1.8× step driven by metadata extraction and the embedding pass), and L2 takes 2,958s (a further 7.3× step driven entirely by the per-file LLM summary call). Metadata extraction and search-engine indexing together account for under 7% of total ingest cost at every level, so the cost question reduces to a single binary: is the per-file summary worth running on this file? The next subsection answers it on the workload Acropolis’s HDF5/netCDF design was built for.

C. Cost–Accuracy Tradeoff of Index Depth

With the cost picture established (§C), the question is whether the L2 summary actually buys retrieval accuracy on the workload Acropolis’s HDF5/netCDF design was built for. We use the 149-file, 67 GB CMIP6 corpus (§A) and sweep both the indexing depth and the retrieval depth k from 1 to 100 across all four QueryEngines and their available retrieval modes.

Figure 4 shows two depth findings. First, L0 → L1 is again essentially flat across all engines and all k , but for the opposite reason as on source code: CMIP6 filenames already encode the file’s full provenance (variable, model, experiment, member, grid, time), so the structural metadata L1 adds is information the path already carried. A consequence is that the format-aware HDF5 `H5Lvisit` extraction (§III-B) is not independently validated as a retrieval lift here; the headline win on CMIP6 is entirely the L2 LLM summary. Second, L1 → L2 is dramatic at small k : across keyword and hybrid engines,

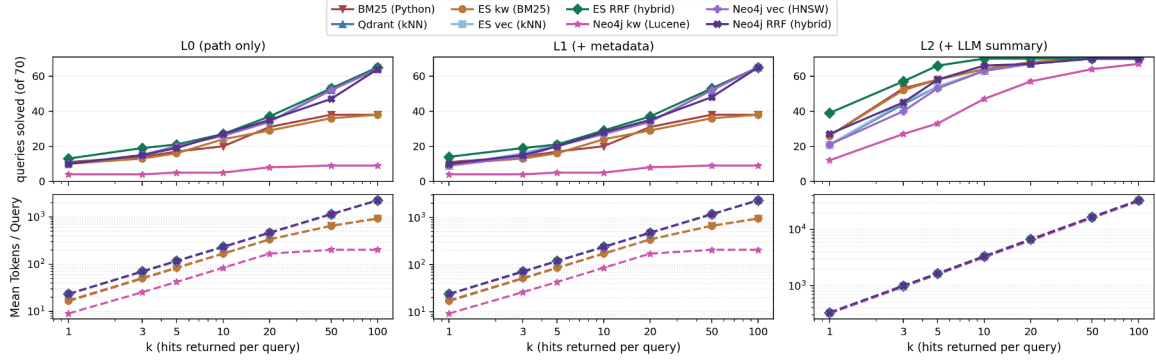


Fig. 4: Indexing-depth and retrieval-depth sweep on the 149-file CMIP6 corpus, across all four QueryEngine implementations (showing each engine’s retrieval modes where applicable). *Top row*: queries solved (of 70) versus retrieval depth k ; *bottom row*: mean LLM-context tokens consumed per query (log scale). L0 $\rightarrow$ L1 is essentially flat because CMIP6 filenames already carry the provenance that L1’s structural metadata would add; the L1 $\rightarrow$ L2 jump is dramatic at small k because only the L2 summary describes files in the query’s own vocabulary, and every engine plateaus by $k \approx 10$ at exponentially growing payload cost. L2 summaries use Qwen2.5-Coder 14B [24]; embeddings via `nomic-embed-text` [26].

top-1 accuracy at L2 is several times higher than at L1, and by $k=5$ the strongest keyword and hybrid engines approach 80% recall where at L1 they had been below 40%. The L2 summary is the first payload in which the file’s contents are described in the same vocabulary the query uses (“near-surface air temperature” rather than the `netCDF` variable name `tas`), which is what the embedding model and the keyword scorer both need to find the file. The sub-40% L1 recall is itself the evidence that these queries are not answerable from the path or structural metadata: a retriever given the full filename and the `H5Lvisit` fields still misses most of them, and only the content-level L2 summary resolves them. L1 can close the gap mechanically through the high- k tail—returning more candidates until the target surfaces—but only at exponentially growing payload cost (bottom row), whereas L2 reaches the same recall at the low- k end of that curve, resolving the semantic queries at far lower per-query token cost.

The sweep also gives the operator a defensible k knee. Across every engine, L2 reaches its accuracy plateau by $k \approx 10$; beyond that, additional candidates return diminishing accuracy at exponentially growing response-payload tokens. The keyword family climbs to the plateau fastest, the dense family climbs slowest, and the hybrid family sits between them, but no engine continues to benefit meaningfully from k beyond the knee. The practical consequence is a clean operator policy: keep L1 as the default for path-heavy formats, promote content-heavy directories to L2 selectively (via `user.acropolis.depth_recursive`) to pay the summary cost only where it changes the answer, and serve queries at $k \approx 10$ to capture the plateau without paying for the exponential tail.

D. Summarizer Model Sensitivity

The summarizer LLM is the dominant ingest cost (§C) and the only stage an operator tunes without touching the storage substrate or the search engine, so we measure how

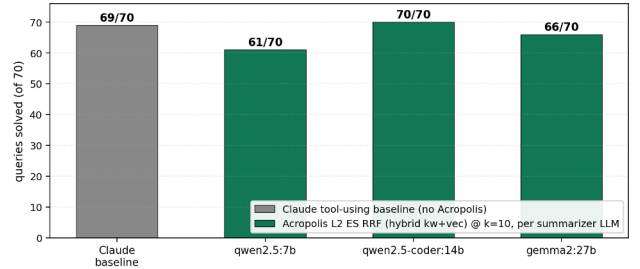


Fig. 5: Retrieval accuracy on the 70-query CMIP6 scientific benchmark for Acropolis at L2 Elasticsearch RRF, $k=10$, varying only the `SummaryOperator`’s ingest LLM across three locally hostable models ($\sim 7\text{B}$ – 27B parameters); the grey bar is the Claude tool-using baseline. A code-specialized 14B model reaches 70/70, matching the baseline at the benchmark ceiling, while a general-purpose 27B model trails by four queries despite being nearly twice the size.

strongly retrieval accuracy depends on it on the workload Acropolis was designed for. Holding the corpus, query set, and Elasticsearch RRF backend fixed at L2 with $k=10$, we regenerate the CMIP6 L2 summaries with three locally hostable models—Qwen2.5 7B [24], Qwen2.5-Coder 14B [24], and Gemma2 27B [27]—and replay the 70-query scientific workload. Figure 5 shows a 9-query spread across the summarizers: Qwen2.5-Coder 14B reaches 70/70, matching the Claude tool-using baseline at the benchmark ceiling (69/70 vs 70/70 is within the benchmark’s resolution); Gemma2 27B reaches 66/70; and Qwen2.5 7B trails at 61/70. The pattern is consistent with specialization mattering more than size: the 14B code-specialized model exceeds the 27B general model by four queries, suggesting the summarizer is a real quality lever and a mid-size domain-specialized model is a reasonable default. 1–4 query differences sit at the benchmark’s resolution (§V);

the 9-query spread (Qwen2.5 7B at 61/70 vs the 14B at 70/70) is the more reliable signal.

E. Summary Length Sensitivity

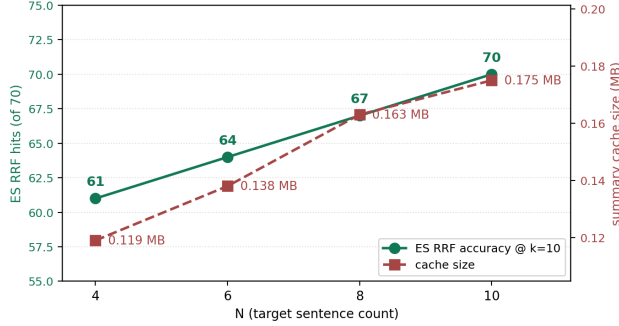


Fig. 6: Acropolis retrieval accuracy on the 70-query CMIP6 benchmark (Elasticsearch RRF at $k=10$, solid green, left axis) and on-disk summary-cache footprint (dashed red, right axis) versus the target summary length N in the SummaryOperator prompt. Both accuracy and footprint grow monotonically with N ; accuracy reaches a perfect 70/70 at $N=10$.

The target summary length N (§ III-C) trades retrieval signal against the on-disk footprint of the indexed summary. Holding everything else fixed and configured at L2 with Elasticsearch RRF at $k=10$, we re-ingest the CMIP6 corpus across target lengths $N \in \{4, 6, 8, 10\}$ sentences and replay the 70-query workload. As Figure 6 shows, accuracy is monotonic in N on this corpus: 61/70 at $N=4$, 64/70 at $N=6$, 67/70 at $N=8$, and 70/70 at $N=10$. Per-step deltas sit at the benchmark’s resolution (§ V); the full $N=4$ to $N=10$ delta (61 vs 70 queries) is the more reliable signal. The on-disk summary footprint grows correspondingly from 0.119 MB at $N=4$ to 0.175 MB at $N=10$, a $\sim 47\%$ increase across the sweep. The practical takeaway is that a richer summary buys more on scientific data than on source code, where the same sweep had been essentially flat: a longer summary lets the embedder and the keyword scorer match against more distinctive vocabulary, so the operator pays a modest footprint cost for higher recall.

F. Weak-Scaling of the QueryEngine

This evaluation characterizes how the QueryEngine’s retrieval surface scales as nodes are added. We run a weak-scaling study from 1 to 8 NCSA Delta [23] nodes, storing 10K Blobs per node with 5% containing a planted “needle” keyword, and time a BM25 semantic_query for “needle” returning the top-10 hits. Figure 7 shows the mean per-query time growing from ~ 243 ms on one node to ~ 482 ms on eight nodes, roughly $2\times$ across an $8\times$ scale-out. The cost grows because the retriever issues a collective broadcast (similar to MPI’s Allreduce) to locate and aggregate candidate hits across all participating nodes before returning the global top-10, so per-query latency tracks the broadcast cost rather than per-node search work.

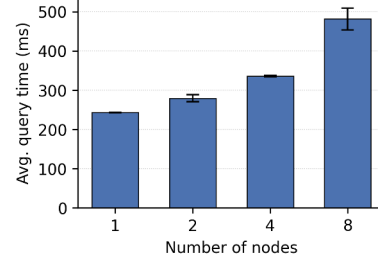


Fig. 7: Weak-scaling of Acropolis BM25 retrieval (10K Blobs per node, 5% containing a planted “needle” keyword; top-10 query). Results demonstrate sub-linear scaling of mean per-query time across 1 to 8 nodes.

G. Agent Tool-Call and Token Savings

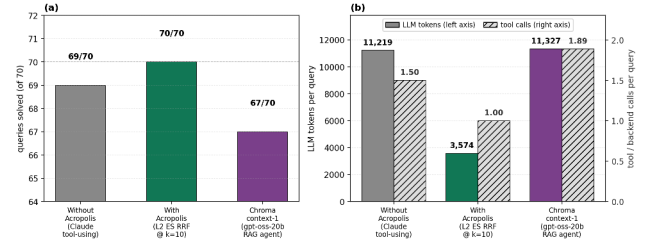


Fig. 8: Three-system comparison on the 70-query CMIP6 scientific benchmark. (a) Queries solved out of 70 for the Claude tool-using baseline (Glob/Grep/Read/Bash), Acropolis (Claude with the semantic_query MCP at L2 Elasticsearch RRF, $k=10$), and the Chroma context-l gpt-oss-20b model [12], [28] run inside a custom RAG harness over the same corpus. (b) Mean LLM tokens per query (left axis, solid bars) and mean tool/backend calls per query (right axis, hatched bars). Acropolis matches the baseline on accuracy and substantially reduces both tool calls and LLM tokens against both alternatives.

In this evaluation, we measure the end-to-end effect of giving an agent Acropolis’s semantic_query tool on the scientific workload Acropolis was designed for. We compare three configurations: the Claude tool-using baseline (no Acropolis; the agent has only the standard shell tools), Acropolis (the same agent with semantic_query added, configured at L2 Elasticsearch RRF with $k=10$), and the released Chroma context-l model (gpt-oss-20b fine-tune [12], [28]) plugged into a custom RAG harness over the same corpus, since running Chroma’s released harness would require duplicating the 67GB CMIP6 corpus into a separate index (the cost-prohibitive workflow § I motivates avoiding), so the comparison evaluates the model itself on a like-for-like substrate. The Chroma-arm token count reflects gpt-oss-20b’s per-turn cost rather than Claude’s, so panel (b)’s Acropolis-vs-Chroma gap mixes architecture and model. We report three axes: queries solved out of 70, mean LLM tokens per query (Claude Code / cost divided by query count), and mean tool/backend calls

per query.

From Figure 8, Acropolis matches the baseline on accuracy and converts the transparent context extraction of C1 and the MCP surfacing of C3 into measured efficiency gains on the other two axes. *Accuracy*: Acropolis reaches 70/70, the baseline 69/70, and Chroma 67/70 — all three sit within a one-query band that is below the benchmark’s resolution (§ V), so we report this axis as a match rather than a win. *Tool calls*: Acropolis averages ≈ 1.00 call per query, while the baseline averages 1.50 (Claude chains shell tools to verify guesses) and Chroma 1.89 (the RAG agent issues multiple retrievals per query); since every ingested file already carries a searchable description, the agent’s first `semantic_query` call typically lands directly on the correct file rather than having to iterate `glob/grep/Read` cycles. *LLM tokens*: Acropolis averages 3,574 tokens per query (Claude Code’s /cost divided by query count, § IV-A) against the baseline’s 11,219 and Chroma’s 11,327, a $3.1\times$ reduction versus the baseline.

The $3.1\times$ token reduction exceeds the $1.5\times$ call reduction because per-turn payloads also differ. Each baseline turn carries call overhead (system prompt, prior results, prior reasoning—typically 6–10 K input tokens—plus ~ 200 –500 output tokens) plus tool-result payload dominated by raw `Read` output: shell-tool turns return whole `netCDF/HDF5` headers and source-tree slices, often kilobytes of opaque metadata. An Acropolis turn returns the ranked top-10 *summaries*—short, query-shaped, vocabulary-aligned. Collapsing ~ 1.5 large-payload baseline turns into one small-payload Acropolis turn reduces both turn count and per-turn cost, which together explain the $3.1\times$ gap. The back-end embedding cost (~ 30 tokens to a ~ 137 M-parameter sentence-embedding model encoding the query for the HNSW+BM25+RRF index) is two orders of magnitude smaller and does not move the headline.

V. DISCUSSION

Limitations: The retrieval evaluation is on a single scientific domain (CMIP6 climate-model output in `netCDF/HDF5`), and the 70-query workload was authored by the same team that assembled the corpus, so per-question generalization to held-out users, other `HDF5`-family layouts (e.g., `NetCDF-CF` for ocean models, neuroimaging `BIDS`, structural biology), and other scientific domains is not directly demonstrated. The benchmark is also small enough that the 1–4 query differences between configurations cannot always be resolved with statistical confidence. The end-to-end retrieval results (§§ IV-C–E, IV-G) are reported on a single CLIO node; the multi-node weak-scaling result (§ IV-F) is a microbenchmark of the BM25 QueryEngine’s broadcast cost on synthetic data rather than an end-to-end run, so claims about multi-tier parallel-filesystem placement should still be read as design intent rather than fully measured behavior. Finally, the dominant per-file ingest cost (the L2 LLM summary, § IV-B) was measured at the thousand-file scale; the cost at the scientific-campaign scales the introduction invokes (hundreds of thousands of `HDF5` files from an ensemble run) is asserted via the operator’s content-

addressable idempotency to converge to extraction cost on re-ingest, but this convergence has not been directly measured.

Future Work: The evaluation surfaces two retrieval-side levers worth pursuing further. The summarizer-model sensitivity result (§ IV-D) shows a 9-query spread on CMIP6 across locally hostable models in the 7–27 B range, with the code-specialized 14 B model reaching the benchmark ceiling; training or fine-tuning a domain-specialized summarizer (climate vocabulary, instrument metadata, units) is the natural extension and is likely necessary to make the same recipe work on harder scientific domains where the off-the-shelf code-specialized models do not already happen to win. The retrieval-depth sweep (§ IV-C) further shows that engine families converge to the plateau at different rates, with the keyword family climbing fastest; a learned engine-selection layer that picks the cheapest engine likely to solve a given query could approach the per-query maximum without paying every engine’s populate cost.

VI. CONCLUSION

We presented Acropolis, a system that closes the agent-driven data-discoverability gap by attaching machine-readable meaning to files at the moment they are written and exposing the resulting search surface to LLM agents through a single typed entry point. The four contributions—a transparent ingest-time summary operator with adaptive depth, a pluggable search-engine abstraction across lexical, vector, and graph engines surfaced as a deployment-time choice and evaluated empirically, an MCP server that exposes the search surface to any compatible agent, and a scientific-data evaluation on a 70-query CMIP6 retrieval benchmark that quantifies the resulting agent-side savings—are realized inside a single CLIO [14] runtime so that no part of the system has to reconstruct, after the fact, knowledge the storage layer already had at write time. On a 70-query CMIP6 scientific benchmark over 149 climate-model files, Claude with Acropolis registered alongside its standard shell tools matches the unaided shell-tool baseline at the benchmark ceiling (70/70 versus 69/70 accuracy, a one-query gap below the benchmark’s resolution) while issuing one-third fewer tool/backend calls per query (1.00 vs 1.50) and consuming roughly $3.1\times$ fewer LLM tokens (3,574 vs 11,219), with no prompt-level instruction to prefer the MCP. The deeper message is architectural: when the layer that sees every write also owns the layer that answers semantic queries, the cost of being useful to LLM agents drops sharply.

ACKNOWLEDGMENT

This material is based upon work supported in part by the National Science Foundation (NSF) under Awards OAC 2411318 and OAC 2313154. This work used DeltaAI at the National Center for Supercomputing Applications (NCSA) through allocation CIS250329 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by NSF grants #2138259, #2138286, #2138307, #2137603, and #2138296. DeltaAI is an advanced computing and data resource supported by NSF (award OAC 2320345) and the State of Illinois, and a joint effort of the University of Illinois Urbana-Champaign and NCSA.

REFERENCES

- [1] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, “Augmenting large language models with chemistry tools,” *Nature Machine Intelligence*, vol. 6, pp. 525–535, 2024.
- [2] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha, “The AI scientist: Towards fully automated open-ended scientific discovery,” 2024, arXiv:2408.06292.
- [3] Chroma, “Chroma: The AI-native open-source embedding database,” <https://www.trychroma.com>, 2024.
- [4] VAST Data, “Introducing VAST vector search and InsightEngine with NVIDIA,” <https://www.vastdata.com>, 2025.
- [5] WEKA, “WEKA introduces NeuralMesh: An auto-vectorizing storage appliance for AI,” <https://www.weka.io>, 2025.
- [6] IBM Research, “Content-aware storage: Filesystem-native vector search with ACL propagation,” <https://research.ibm.com>, 2025.
- [7] DDN, “DDN enterprise AI HyperPOD with NVIDIA NeMo retriever,” <https://www.ddn.com>, 2025.
- [8] Anthropic, “Model context protocol specification,” <https://modelcontextprotocol.io>, 2024.
- [9] A. Rajasekar, R. Moore, C.-Y. Hou, C. A. Lee, R. Marciano, A. de Torcy, M. Wan, W. Schroeder, S.-Y. Chen, L. Gilbert, P. Tooby, and B. Zhu, *iRODS Primer: Integrated Rule-Oriented Data System*. Morgan & Claypool, 2010.
- [10] I. Foster, “Globus online: Accelerating and democratizing science through cloud-based services,” *IEEE Internet Computing*, vol. 15, no. 3, pp. 70–73, 2011.
- [11] M. Folk, G. Heber, Q. Koziol, E. Pourmal, and D. Robinson, “An overview of the HDF5 technology suite and its applications,” in *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases*, 2011, pp. 36–47.
- [12] Chroma, “context-1: A gpt-oss-20b fine-tune for context engineering,” <https://www.trychroma.com/context-1>, 2025.
- [13] Y. Shi *et al.*, “LSFS: An LLM-based semantic file system for AIOS,” in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.11843.
- [14] IOWarp Project, “CLIO: A multi-tiered I/O system for AI and scientific workloads,” <https://iowarp.ai>; source: <https://github.com/iowarp>, 2024.
- [15] libfuse Project, “libfuse: Filesystem in userspace (FUSE) reference implementation for the Linux kernel,” <https://github.com/libfuse/libfuse>, 2024.
- [16] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [17] C. Gormley and Z. Tong, *Elasticsearch: The Definitive Guide*. O’Reilly Media, 2015.
- [18] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
- [19] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759.
- [20] Qdrant Team, “Qdrant: Vector database for the next generation of AI applications,” <https://qdrant.tech>, 2024.
- [21] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*, 2nd ed. O’Reilly Media, 2015.
- [22] Amazon Web Services, “ESGF-world CMIP6 collection, registry of open data on AWS,” <https://registry.opendata.aws/esgf-world/>, 2024, s3://esgf-world/CMIP6/.
- [23] National Center for Supercomputing Applications, “Delta GPU computing system,” <https://delta.ncsa.illinois.edu/>, 2024, NCSA Delta is an NSF-funded GPU-accelerated HPC system at the University of Illinois Urbana-Champaign.
- [24] Qwen Team, “Qwen2.5: A party of foundation models,” <https://qwenlm.github.io/blog/qwen2.5/>, 2024.
- [25] Ollama, “Ollama: Get up and running with large language models,” <https://ollama.com>, 2024.
- [26] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” 2024, arXiv:2402.01613.
- [27] Gemma Team, Google DeepMind, “Gemma 2: Improving open language models at a practical size,” 2024, arXiv:2408.00118.
- [28] OpenAI, “gpt-oss-20b model card,” <https://huggingface.co/openai/gpt-oss-20b>, 2024.
- [29] K.-K. Muniswamy-Reddy, D. A. Holland, U. Braun, and M. Seltzer, “Provenance-aware storage systems,” in *USENIX Annual Technical Conference (ATC)*, 2006.
- [30] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eysers, M. Seltzer, and J. Bacon, “Practical whole-system provenance capture,” in *Proceedings of the Symposium on Cloud Computing (SoCC)*, 2017, pp. 405–418.